

Manual Completo do Ecossistema AIDD

Guia Prático End-to-End: Da Tríade Canônica ao Deploy em Produção

Ecossistema AIDD — heverton-dev

19 de Setembro de 2026

Sumário

Seção 1 — Visão Geral e Arquitetura Canônica	3
O que é o Ecossistema AIDD?	3
As 11 Leis Invioláveis do Ecossistema AIDD	3
As 8 Ferramentas Especializadas	4
A Tríade Canônica de Criação (Os 3 Fluxos de Ponta a Ponta)	5
Detalhamento dos 3 Fluxos:	5
Os 24 Quality Gates do Ecossistema	6
Seção 2 — Pré-requisitos, Instalação e Configuração	8
Requisitos do Sistema	8
Diagnóstico Instantâneo do Host (<code>preflight-host</code>)	8
Instalação e Auto-Bootstrap	8
Seção 3 — O Projeto Prático: Construindo o “iTask”	10
Funcionalidades do iTask:	10
Seção 4 — Caminho 1: Execução via Terminal (CLI Central)	11
Etapa 1: Disparo do Fluxo Canônico	11
Etapa 2: Adição de Novas Fatias Verticais (VSA)	11
Etapa 3: Injeção de Segurança e Componentes Certificados	11
Etapa 4: Auditoria de Conformidade	11
Seção 5 — Caminho 2: Execução via Assistentes de IA (Universalidade)	12
Padrão Canônico de Skills (<code>aidd-*</code> , máx. 3 palavras)	12
Interoperabilidade Universal dos Slash Commands	13
Seção 6 — Desacoplamento Low-Code: Guia Completo do Bridge	14
Ciclo de Execução do Bridge:	14
Seção 7 — Referência Rápida e Tabela de Comandos	15
Comandos da CLI Central (<code>ecossistema.py</code>)	15
Seção 8 — Glossário Técnico do Ecossistema	16

Seção 1 — Visão Geral e Arquitetura Canônica

O que é o Ecossistema AIDD?

O **Ecossistema AIDD (AI-Driven Development)** é uma plataforma industrial de engenharia de software governada por inteligência artificial determinística. Ele transforma uma ideia bruta ou especificação de negócio em software testado, tipado, modular e pronto para produção com **Zero Stubs**, **Zero Vibe Coding** e **Governança Criptográfica SHA-256**.

Funciona como uma fábrica de montagem em esteira de alta precisão: cada ferramenta é uma estação de trabalho especializada conectada por contratos formais de handoff, Quality Gates binários (exit 0 = passa, exit 1 = bloqueia) e paridade estrita entre terminais CLI e múltiplos assistentes de IA (harnesses).

As 11 Leis Invioláveis do Ecossistema AIDD

1. **Determinism First:** Uso de scripts determinísticos, AST, regex ou JSON Schema para qualquer tarefa mecânica. O LLM nunca executa operações críticas sem travas formais.
2. **Binary Quality:** Todo commit ou evolução é submetido à bateria de Quality Gates (`python ecossistema.py audit`). Se um gate falhar, o build é sumariamente abortado.
3. **Structured Persistence:** O estado é persistido em arquivos de auditoria estruturados (JSON, SQLite WAL), nunca na memória volátil da sessão do chat.
4. **Extreme Token Economy:** Prompts compactos, regras sem redundância e diretivas em estilo conciso para mitigar o estouro de contexto.
5. **Zero Stubs / Zero Mocks:** Código 100% funcional, estritamente tipado, com cobertura de testes reais (unitários, integração e propriedades).
6. **Agnostic Supremacy:** Zero lock-in de fornecedor, sistema operacional (Windows, Linux, macOS), LLM ou harness de IA.
7. **Developer in Control:** Execuções síncronas e transparentes com paradas obrigatórias para aprovação humana entre cada grande etapa.
8. **Label Honesty:** Proibição de alegações de conformidade, certificações ou notas além dos dados reais mensurados pelas ferramentas.
9. **Tool Testing Discipline:** Ciclo contínuo de testes end-to-end com auto-correção, commit limpo e relatórios factuais em `docs/teste-end-to-end/`.
10. **Quarteto Sine Qua Non Dinâmico:** Todo projeto gerado ou evoluído no ecossistema nasce nativamente com 4 pilares:
 - **Swagger Studio** (`/swagger` ou `/docs`): Contratos OpenAPI 3.1 interativos.
 - **Webhook Studio** (`/webhooks`): Ingestão, disparo e assinatura HMAC de webhooks.

- **MCP Studio** ([/mcp](#)): Exposição de Model Context Protocol para consumo por agentes.
 - **Guia do Utilizador** ([/docs/guia](#) ou [/guia](#)): Documentação operacional completa e dinâmica.
11. **Padrão-Ouro de Stack Tecnológica:** Frontend gerado obrigatoriamente em **Next.js + TypeScript + Tailwind CSS**; Backend em **Python puro + SQLite WAL** (ou PostgreSQL assíncrono via `asyncpg` para projetos unificados); API em **OpenAPI 3.1**.
-

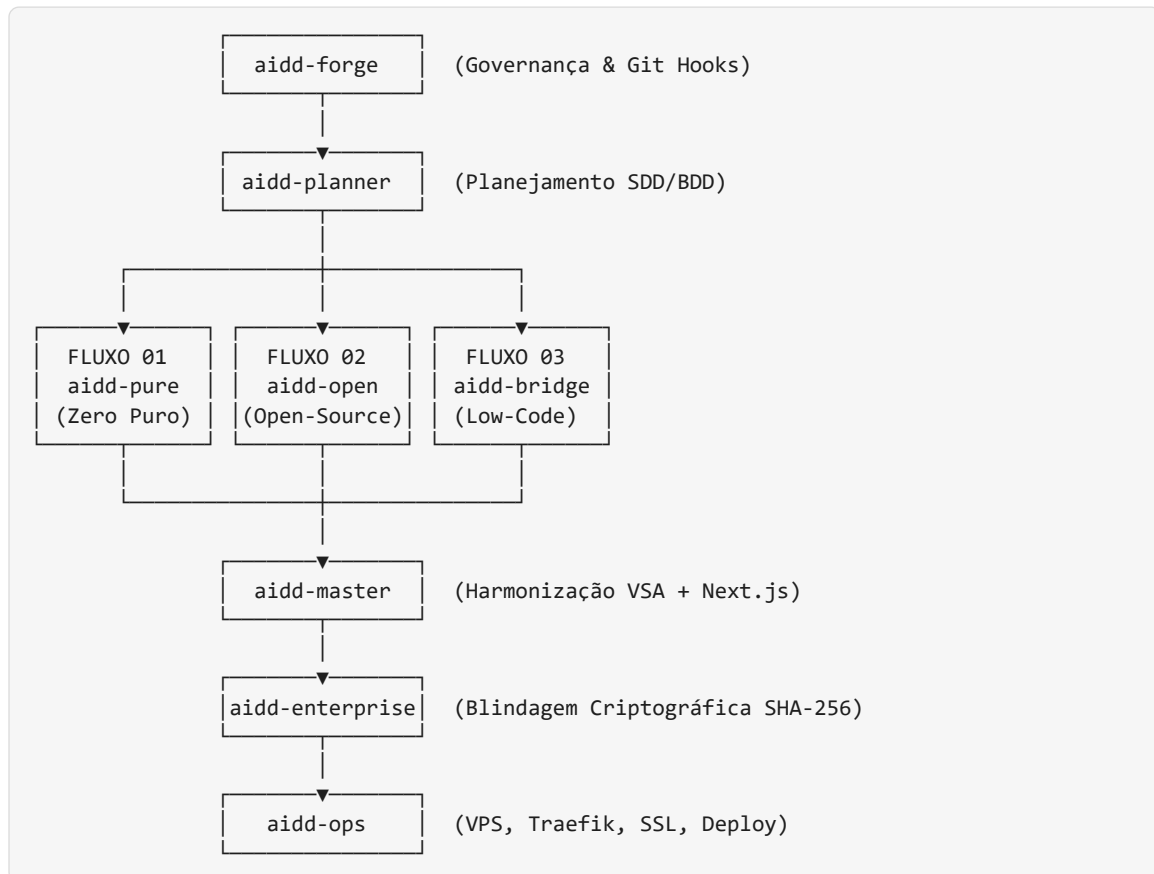
As 8 Ferramentas Especializadas

O ecossistema é composto por 8 ferramentas integradas em [tools/](#) :

#	Ferramenta	Especialidade e Função	Comando CLI Direto
1	aidd-forge	Bootstrap de governança, blindagem de hooks git, isolamento de fases e regras	<code>python ecossistema.py forge init</code>
2	aidd-planner	Motor de intake e planeamento BDD/SDD; gera e valida o PLANNER.json	<code>python ecossistema.py planner init</code>
3	aidd-generator	Fábrica autônoma de software com pipeline de 8 fases determinísticas e TDD Red-Green	<code>python ecossistema.py generate</code>
4	aidd-factory	Curadoria de motores open-source de ponta, fatiamento vertical e Docker Compose	<code>python ecossistema.py factory</code>
5	aidd-bridge	Erradicação de vendor lock-in (Lovable, v0, Bolt), migração de banco para PostgreSQL	<code>python ecossistema.py bridge</code>
6	aidd-master	Harmonização em Monólito Modular VSA (Vertical Slice Architecture) e Next.js	<code>python ecossistema.py master</code>
7	aidd-enterprise	Injeção de componentes corporativos certificados SHA-256 e auditoria anti-drift	<code>python ecossistema.py enterprise</code>
8	aidd-ops	Meta-orquestrador agêntico de infraestrutura: VPS, Traefik, SSL, Docker e Uptime Kuma	<code>python ecossistema.py ops</code>

A Tríade Canônica de Criação (Os 3 Fluxos de Ponta a Ponta)

Toda nova aplicação no ecossistema nasce a partir de `aidd-forge` (governança) e `aidd-planner` (intake interativo), derivando em 3 caminhos de alta especialização:



Detalhamento dos 3 Fluxos:

- FLUXO 01 — `aidd-pure` (Do Zero Puro | Slash: `/pure`):**
 - **Objetivo:** Geração de código autoral sob medida do zero absoluto.
 - **Engine:** `aidd-generator` via TDD estrito Red-Green e arquitetura modular limpa.
 - **Acionamento:** `/pure <ideia> [dominio]`
OU `python ecossistema.py pure --nome <n> --slug <s> --dominio <d> --pasta <p>`
- FLUXO 02 — `aidd-open` (Motores Open-Source | Slash: `/open`):**
 - **Objetivo:** Criação ultra-acelerada reutilizando motores open-source consagrados (ex.: Chatwoot, Cal.com, Twenty CRM, Authentik).
 - **Engine:** `aidd-factory` gerando gateways FastAPI, orquestração multi-service e fatias verticais VSA acopladas.
 - **Acionamento:** `/open <ideia> [dominio]`
OU `python ecossistema.py open --nome <n> --slug <s> --dominio <d> --pasta <p>`

3. FLUXO 03 — `aidd-bridge` (Low-Code / Apps Unificadas | Slash: `/bridge`):

- **Objetivo:** Libertação de apps prototipados no Lovable, v0 ou Bolt para VPS própria sem custos de nuvem proprietária.
- **Engine:** `aidd-bridge` realizando scan anti-lockin, remoção de dependências Supabase/BaaS, migração para PostgreSQL e preservação total da UI/UX original.
- **Acionamento:** `/bridge <ideia> --origem <pasta>` ou
`python ecossistema.py bridge --nome <n> --slug <s> --dominio <d> --pasta <p> --origem <o>`

Os 24 Quality Gates do Ecossistema

O comando universal `python ecossistema.py audit` executa os 24 Quality Gates de forma estrita. Exit 0 certifica aprovação total; qualquer falha impede o avanço.

#	Quality Gate	Escopo de Verificação
1	<code>G_ECOSSISTEMA_INTEGRIDADE</code>	Estrutura íntegra de <code>tools/</code> , <code>AGENTS.md</code> , READMEs e ausência de pastas <code>.git</code> acidentais
2	<code>G_ORQUESTRADOR_SINCRONO</code>	Validação de contratos JSON Schema da Tríade e conformidade do orquestrador
3	<code>G_HARNESS_COMPAT</code>	Sincronismo perfeito dos artefatos em todos os assistentes de IA suportados
4	<code>G_COMPONENTE_AGNOSTICO</code>	Cobertura multi-harness (skills, comandos, hooks) conforme o manifesto
5	<code>G_ZERO_HEADLESS</code>	Proibição de subagentes concorrentes em segundo plano sem controle humano
6	<code>G_INFRA_COMPOSE</code>	Validação estática de Docker Compose e scripts de banco via Hadolint e Checkov
7	<code>G_HADOLINT</code>	Inspeção de boas práticas OCI em 100% dos Dockerfiles do repositório
8	<code>G_TESTES_REAIS</code>	Execução real de pytest em todas as ferramentas com orçamento de skipped travado
9	<code>G_DEPENDENCIAS_PIN_HASH</code>	Fixação estrita de versões e hashes SHA-256 criptográficos em <code>requirements.lock</code>
10	<code>G_FRONTEND_LAYERS</code>	Isolamento estrito entre componentes visuais de UI e camadas de rede no Frontend
11	<code>G_ISOLATION_AUDIT</code>	Isolamento vertical estrito entre fatias de domínio VSA sem vazamento de acoplamento

#	Quality Gate	Escopo de Verificação
12	G_PROTOCOL_FALLBACK	Paridade de contratos e fallback entre chamadas REST e Model Context Protocol (MCP)
13	G_LLM_PROMPT_SHIELD	Blindagem ativa de prompts e sanitização contra ataques de Prompt Injection
14	G_DRIFT_ANALYZER	Deteção de redundâncias funcionais e divergências estruturais inter-fatias
15	G_PROTOTYPE_REWRITE	Isolamento estrito de código sandbox e proibição de promoção sem suíte TDD
16	G_CLI_HELP_CONSISTENCIA	Paridade absoluta entre docstrings/mensagens de ajuda e as flags reais de CLI
17	G_SEGREDOS	Varredura profunda contra credenciais, tokens ou chaves privadas no código
18	G_DRIFT_NUCLEO_COMPARTILHADO	Byte-identidade do núcleo compartilhado entre fonte e destinos
19	G_HONESTIDADE_ROTULO	Verificação de conformidade entre rótulos técnicos e evidências reais
20	G_ARQUITETURA_DELIVERABLE	Aderência às camadas de Clean Architecture no software gerado
21	G_ESCRITOR_ATOMICO	Garantia de escritas de arquivo atômicas sem corrupção em falhas de processo
22	G_TRANSACTION_LOG_LRU	Validação de log de transações e expurgo de memória preventiva
23	G_UNIVERSAL_HARNESS	Garantia de que novos componentes rodam de forma idêntica em 7 harnesses
24	G_PREFLIGHT_HOST	Diagnóstico instantâneo (< 2s) de binários essenciais do sistema operacional

Seção 2 — Pré-requisitos, Instalação e Configuração

Requisitos do Sistema

Componente	Versão Mínima	Finalidade	Comando de Checagem
Python	3.10 ou superior	Motor de execução principal	<code>python --version</code>
Git	2.30 ou superior	Controle de versão e worktrees efêmeras	<code>git --version</code>
Node.js	20 LTS (recomendado)	Construção do Frontend Next.js	<code>node --version</code>
Docker	Qualquer moderno	Containerização local e deploy de produção	<code>docker --version</code>

Diagnóstico Instantâneo do Host (`preflight-host`)

O ecossistema dispõe de um inspetor de ambiente de alto desempenho:

```
# Diagnóstico rápido de dependências locais
python ecossistema.py preflight-host

# Diagnóstico com correção assistida multi-OS
python ecossistema.py preflight-host --fix --dry-run
```

Instalação e Auto-Bootstrap

```
# 1. Clonar o repositório canônico
git clone https://github.com/heverton-dev/ecossistema-aidd.git
cd ecossistema-aidd

# 2. Verificar o estado de saúde do ecossistema
python ecossistema.py status

# 3. Executar o bootstrap assistido de dependências
python ecossistema.py dependencia bootstrap
```

Seção 3 — O Projeto Prático: Construindo o “iTask”

Para demonstrar o funcionamento prático de ponta a ponta, construímos o **iTask** — uma plataforma corporativa de gestão de projetos e tarefas com autenticação segura, Next.js no Frontend, VSA no Backend e deploy em VPS.

Funcionalidades do iTask:

- **Autenticação:** Login, registro e renovação via JWT criptografado com Argon2id.
 - **Módulo de Projetos:** CRUD completo com controle de membros e papéis.
 - **Módulo de Tarefas:** Quadro de tarefas com status, prazos, prioridades e tags.
 - **Módulo de Dashboard:** Métricas agregadas de produtividade e taxa de conclusão.
 - **Quarteto Sine Qua Non Integrado:** Swagger Studio, Webhook Studio, MCP Studio e Guia do Usuário.
-

Seção 4 — Caminho 1: Execução via Terminal (CLI Central)

Etapa 1: Disparo do Fluxo Canônico

Para criar o projeto do zero puro via Fluxo 01 com TDD:

```
python ecossistema.py pure \
  --nome "iTask Platform" \
  --slug itask \
  --dominio gestao \
  --pasta ./projetos/itask
```

(Para simulação sem escrita no disco, adicione a flag `--dry-run`).

O orquestrador síncrono executa: 1. `aidd-forge`: Injeta governança, regras em `AGENTS.md` e git hooks de proteção. 2. `aidd-planner`: Mapeia entidades e valida o contrato `handoff-planner-to-engine.schema.json`. 3. `aidd-generator`: Executa a esteira TDD gerando o domínio, use cases e testes. 4. `aidd-master`: Empacota o Monólito Modular VSA + Next.js com Tailwind CSS. 5. `aidd-enterprise`: Injeta componentes criptográficos certificados SHA-256. 6. `aidd-ops`: Valida Dockerfile, compose e topologia de deploy.

Etapa 2: Adição de Novas Fatias Verticais (VSA)

Com o projeto criado, o desenvolvimento incremental é feito adicionando fatias verticais desacopladas:

```
cd projetos/itask

# Adicionar fatia de faturamento / assinaturas
python ecossistema.py master add-module faturamento --pasta ./projetos/itask
```

Etapa 3: Injeção de Segurança e Componentes Certificados

```
python ecossistema.py enterprise inject skill auth-jwt --dir ./projetos/itask
python ecossistema.py enterprise verificar-drift --dir ./projetos/itask
```

Etapa 4: Auditoria de Conformidade

```
python ecossistema.py audit
```

Seção 5 — Caminho 2: Execução via Assistentes de IA (Universalidade)

No ecossistema AIDD, você pode interagir utilizando qualquer ferramenta de IA: **Google Antigravity, Claude Code, Cursor, OpenCode, MiMoCode ou Gemini CLI.**

Padrão Canônico de Skills (`aidd-*` , máx. 3 palavras)

Todas as habilidades do ecossistema seguem o padrão de nomenclatura concisa com no máximo 3 palavras:

Comando Slash	Skill Canônica Subjacente	O que executa
<code>/pure <ideia></code>	<code>aidd-pure</code>	Executa o Fluxo 01 (Do Zero Puro via TDD)
<code>/open <ideia></code>	<code>aidd-open</code>	Executa o Fluxo 02 (Motores Open-Source)
<code>/bridge <ideia></code>	<code>aidd-bridge</code>	Executa o Fluxo 03 (Desacoplamento Low-Code)
<code>/forge [caminho]</code>	<code>aidd-forge-runner</code>	Bootstrap de governança e blindagem
<code>/planner [cmd]</code>	<code>aidd-planner-runner</code>	Intake BDD/SDD e planejamento estruturado
<code>/master <modulo></code>	<code>aidd-master-runner</code>	Criação de fatia vertical desacoplada
<code>/enterprise</code>	<code>aidd-enterprise-runner</code>	Injeção de módulos certificados SHA-256
<code>/ops [requisito]</code>	<code>aidd-ops-runner</code>	Planejamento e orquestração de infraestrutura

Comando Slash	Skill Canônica Subjacente	O que executa
<code>/melhoria <pedido></code>	<code>aidd-melhoria</code>	Análise profunda pré-código de melhorias
<code>/plan <nome></code>	<code>aidd-plan</code> / <code>aidd-planos</code>	Geração de planos formais em docs/planos/
<code>/orchestrate</code>	<code>aidd-orchestrate</code> / <code>aidd-orca</code>	Execução de planos via worktrees efêmeras

Interoperabilidade Universal dos Slash Commands

Em harnesses que não oferecem menu autônomo de comandos slash na interface gráfica (como Google Antigravity e Gemini CLI), o desenvolvedor pode simplesmente digitar `/pure`, `/open` ou `/bridge` como texto normal no chat. O agente intercepta imediatamente o comando através das regras ativas de `AGENTS.md` e dispara a esteira determinística correspondente.

Seção 6 — Desacoplamento Low-Code: Guia Completo do Bridge

O `aidd-bridge` liberta aplicações prototipadas em ferramentas como **Lovable**, **v0** e **Bolt**, migrando-as para uma infraestrutura autônoma em VPS própria com banco PostgreSQL puro.

Ciclo de Execução do Bridge:

```
# 1. Mapeamento estrutural do protótipo exportado
python ecossistema.py bridge scan ./export-lovable

# 2. Transpilação do banco (Supabase/BaaS -> PostgreSQL puro + migrations)
python ecossistema.py bridge convert-db ./export-lovable

# 3. Migração de autenticação com senhas preservadas
python ecossistema.py bridge migrate-auth \
  --source postgres://supabase:5432/db \
  --target postgres://minha-vps:5432/db

# 4. Empacotamento para Docker + Nginx SSL com identidade visual 100% preservada
python ecossistema.py bridge pack ./export-lovable --domain itask.minhaempresa.com
```

Seção 7 — Referência Rápida e Tabela de Comandos

Comandos da CLI Central (`ecossistema.py`)

```
# Diagnóstico e Auditoria
python ecossistema.py status          # Exibe ferramentas e skills ativas
python ecossistema.py audit          # Executa os 24 Quality Gates
python ecossistema.py preflight-host  # Diagnóstico de ferramentas locais

# Triade Canônica de Criação
python ecossistema.py pure --nome <n> --slug <s> --dominio <d> --pasta <p>
python ecossistema.py open --nome <n> --slug <s> --dominio <d> --pasta <p>
python ecossistema.py bridge --nome <n> --slug <s> --dominio <d> --pasta <p> --origem <o>

# Gestão de Componentes Multi-Harness
python ecossistema.py components sync --tipo todos # Sincroniza em todos os harnesses
python ecossistema.py components verify --tipo todos # Verifica integridade SHA-256

# Dependências Externas do Agente
python ecossistema.py dependencia bootstrap          # Instala skills e MCPs externos
python ecossistema.py dependencia verify            # Audita hashes das dependências
```

Seção 8 — Glossário Técnico do Ecosystema

- **VSA (Vertical Slice Architecture):** Arquitetura que organiza o código por funcionalidades de negócio completas (fatias verticais com domínio, banco e API juntos), em vez de camadas técnicas horizontais.
- **Harness:** Qualquer interface, IDE ou ferramenta de linha de comando onde um agente de IA opera (ex.: Antigravity, Claude Code, Cursor, OpenCode, Gemini CLI).
- **Quality Gate:** Script determinístico de aprovação binária que valida regras arquiteturais e segurança antes de permitir modificações no repositório.
- **Quarteto Sine Qua Non:** Os 4 pilares obrigatórios de todo software gerado: `/swagger`, `/webhooks`, `/mcp` e `/docs/guia`.
- **Zero Stubs:** Princípio inviolável que proíbe funções vazias, `TODO` ou mocks simulados em código de produção.